INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

SBT-DF203 · PRACTICAL LABORATORIES · LAB 1
HTTP ANALYSIS USING WIRESHARK: TEXT TRAFFIC

STUDENT NAME: IBRAHIM ISHAKU
STUDENT ID: 2025/FWSD/11334

FELLOWSHIP IN WEB APPLICATION SECURITY & DIGITAL FORENSICS

COURSE: BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS
HTTP ANALYSIS USING WIRESHARK: TEXT TRAFFIC
INSTRUCTOR: AMINU IDRIS, AMCPN

8TH SEPTEMBER, 2026

ACKNOWLEDGEMENT

I thank my instructor for the comprehensive HTTP Analysis Using Wireshark materials that formed the foundation for this lab. The practical walkthrough on TCP three-way handshake, HTTP request/response analysis, packet capture, and network forensic timeline construction gave me the essential skills required for this practical exercise. I also appreciate the International Cybersecurity and Digital Forensics Academy (ICDFA) for providing the lab environment and resources for this practical assessment.

CERTIFICATION / DECLARATION

I, **Ibrahim Ishaku**, with Student ID: **2025/FWSD/11334**, hereby declare that this lab report titled **"HTTP Analysis Using Wireshark: Text Traffic"** is my original work. All packet captures, traffic analysis, TCP handshake examination, and HTTP request/response reconstruction tasks documented in this report were performed by me in the ICDFA Kali Linux lab environment. The answers are based on my actual practical work and understanding of the course materials. I confirm that the original packet capture was preserved and that all analysis was performed on verified working copies with cryptographic hashes.

Student **Signature:** _____

Date: 7th September, 2026

INSTRUCTOR APPROVAL PAGE

Instructor Name: AMINU IDRIS

Comments / Evaluation:

Instructor Signature: _____

Date: _____

TABLE OF CONTENTS

EXECUTIVE SUMMARY

This lab report documents my practical work on HTTP Analysis Using Wireshark: Text Traffic as required for the SBT-DF203 Basic Networking Skills for Digital Forensics course. I completed a comprehensive network forensic analysis exercise using a local Apache web server and Wireshark/tshark packet capture. I created a local training webpage containing my name and registration number, verified the Apache service, captured the HTTP session traffic, and analysed the TCP three-way handshake (SYN, SYN-ACK, ACK). I examined the HTTP GET request and response, reconstructed the complete TCP conversation using Follow TCP Stream, and documented the connection termination. I prepared a detailed forensic timeline, calculated SHA-256 hashes of the capture files, and identified the forensic value of each protocol layer. Through this lab, I gained practical skills in network traffic analysis, HTTP/TCP forensics, and evidence documentation essential for digital forensics professionals.

1.0 Section 1 – Introduction

Network forensics is the practice of capturing, recording, and analysing network traffic to investigate security incidents and gather digital evidence. This lab focuses on plaintext HTTP traffic, which is unencrypted and therefore fully visible to a network analyst.

Key concepts covered in this lab:

- **HTTP (Hypertext Transfer Protocol)** – Application layer protocol used for web traffic.
- **TCP (Transmission Control Protocol)** – Transport layer protocol that provides reliable, ordered data delivery.
- **TCP Three-Way Handshake** – SYN, SYN-ACK, ACK packets that establish a connection.
- **Encapsulation** – Data is wrapped in headers at each layer (Application → Transport → Network → Data Link).

The lab uses a local Apache web server and the loopback interface (lo) to capture traffic. A separate two-VM setup is recommended for observing Ethernet MAC addresses.

2.0 Section 2 – Lab Folder Structure and Evidence Preparation

2.1 Create the Lab Folder Structure

```
mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab1
pwd
find . -maxdepth 1 -type d -print
```

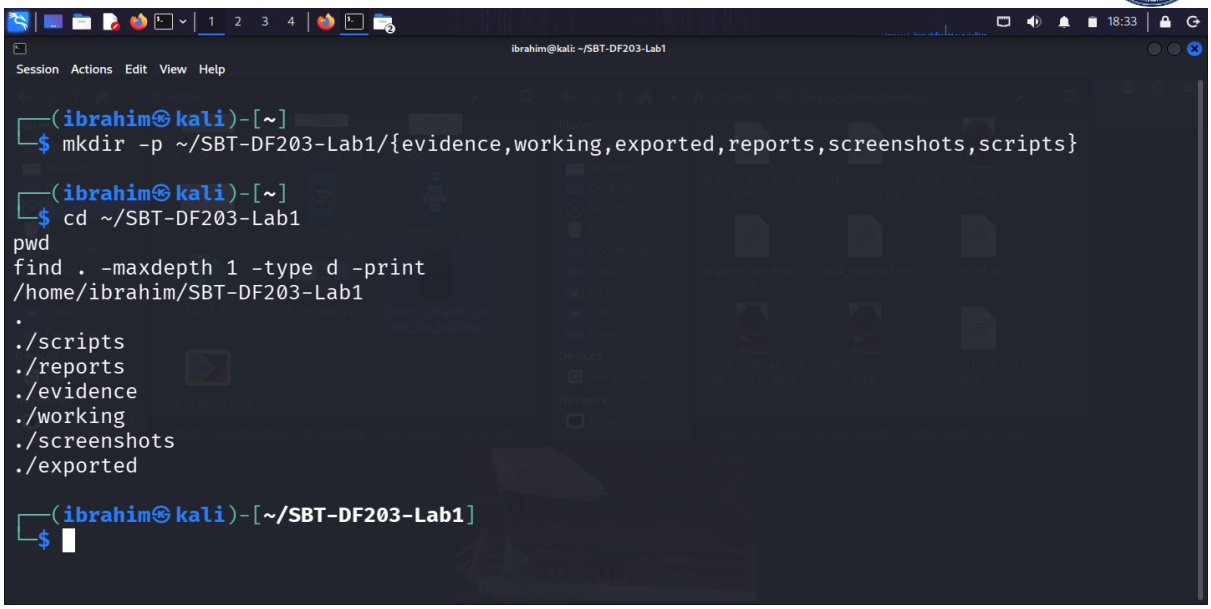


Figure 1.1: Lab folder structure created successfully.

2.2 Install Required Tools

```
sudo apt update
sudo apt install -y apache2 curl wireshark tshark
sudo systemctl enable --now apache2
```

2.3 Create the Webpage

```
printf '<!DOCTYPE html>\n<html>\n<body>\n<h1>SBT-DF203 HTTP Evidence
</h1>\n<p>NAME: Ibrahim Ishaku</p>\n<p>Reg NO: 2025/FWSD/11334</p>\n
</body>\n</html>' | sudo tee /var/www/html/basic.html
```

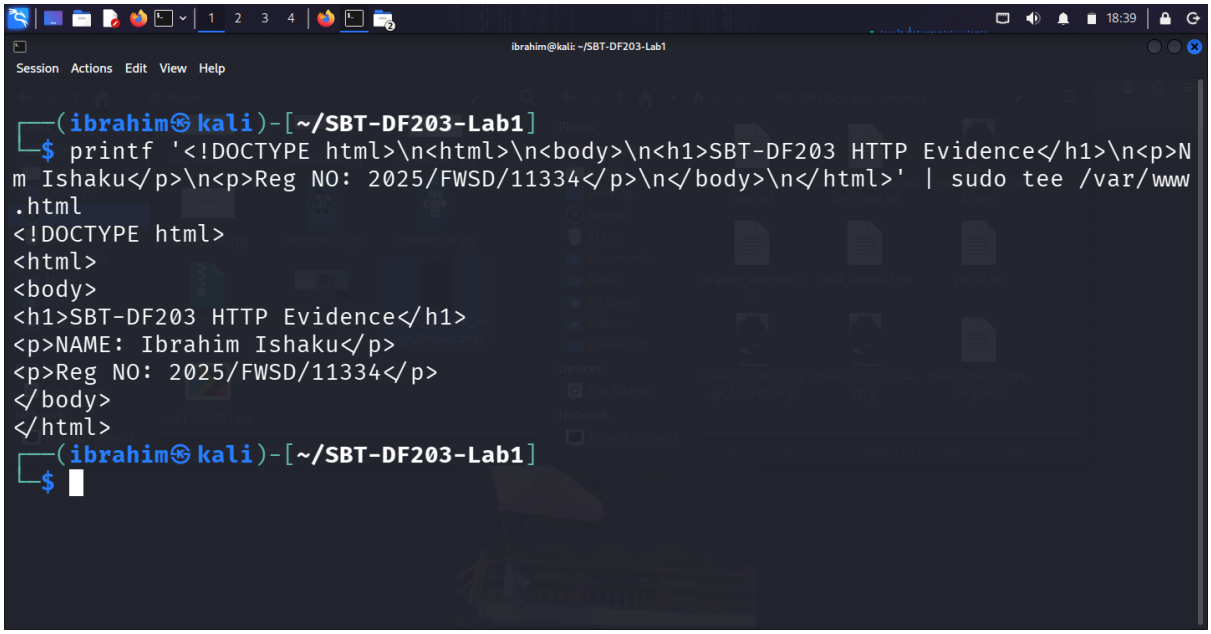


Figure 1.2: Webpage created with student identity.

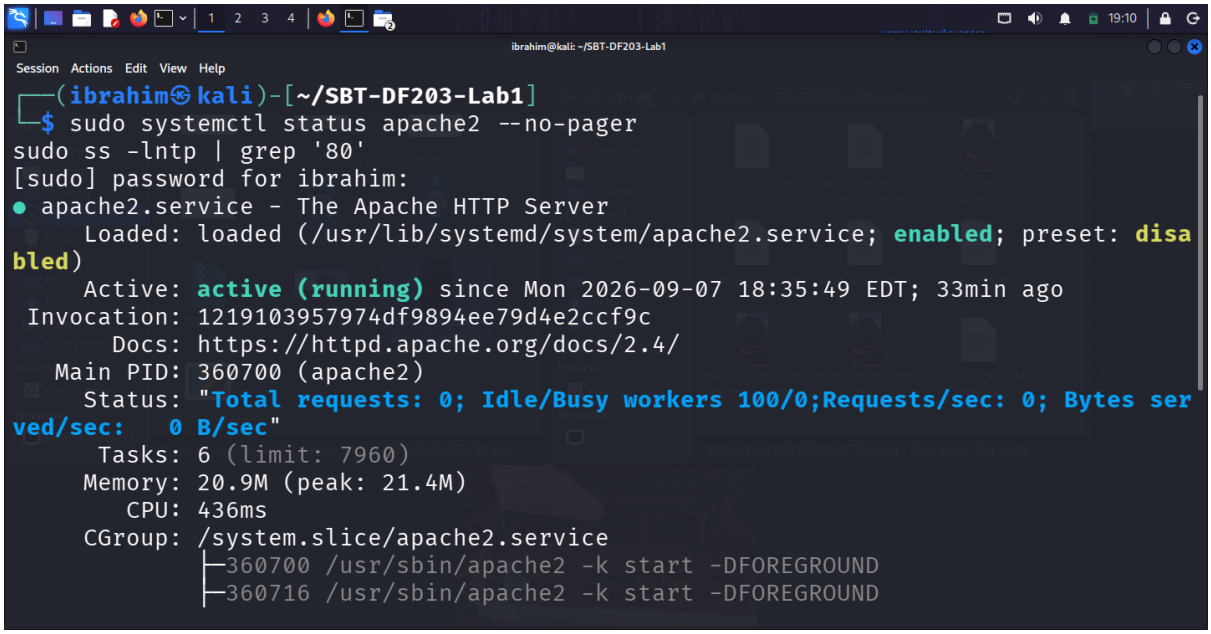
2.4 Mini Chain-of-Custody / Evidence Worksheet

Field	Value
Case/Lab Identifier	SBT-DF203-Lab1-Ibrahim-Ishaku
Trainee Name	Ibrahim Ishaku
Date and Time Started	7th September, 2026
Evidence File Name(s)	basic.pcapng
Source or Generation Method	Local Apache capture on lo interface
Original SHA-256	[Hash calculated after capture]

3.0 Section 3 – Part A: Verify the Web Service and Generate Traffic

3.1 Verify Apache Service

sudo systemctl status apache2 --no-pager
sudo ss -lntp | grep '80'



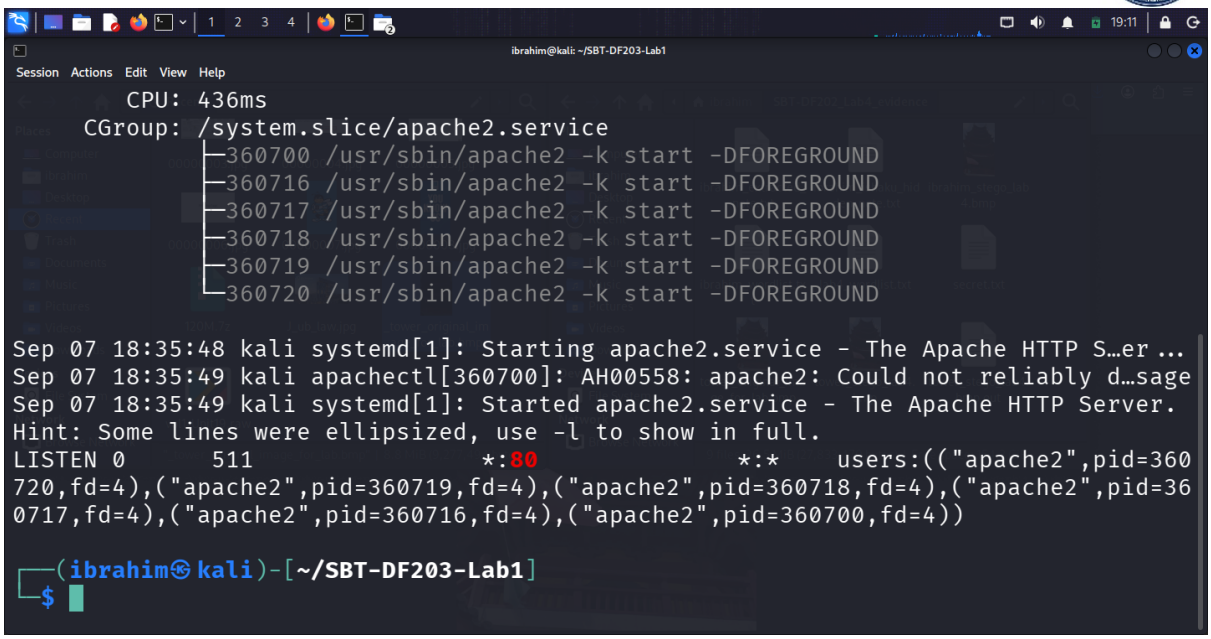


Figure 2.1: Apache service listening on TCP port 80.

3.2 View the Webpage

Open http://127.0.0.1/basic.html in a browser.

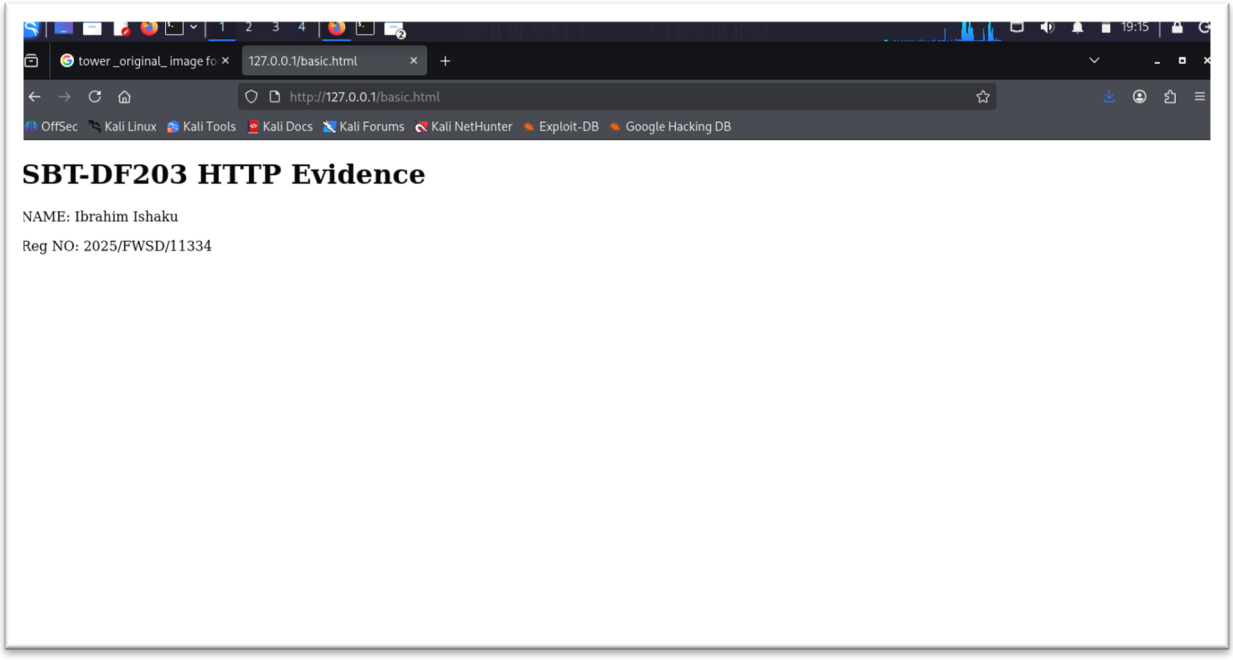


Figure 2.2: Webpage displaying student name and registration number.

3.3 Generate HTTP Request with curl

curl -v http://127.0.0.1/basic.html 2>&1 | tee reports/curl_verbose.txt

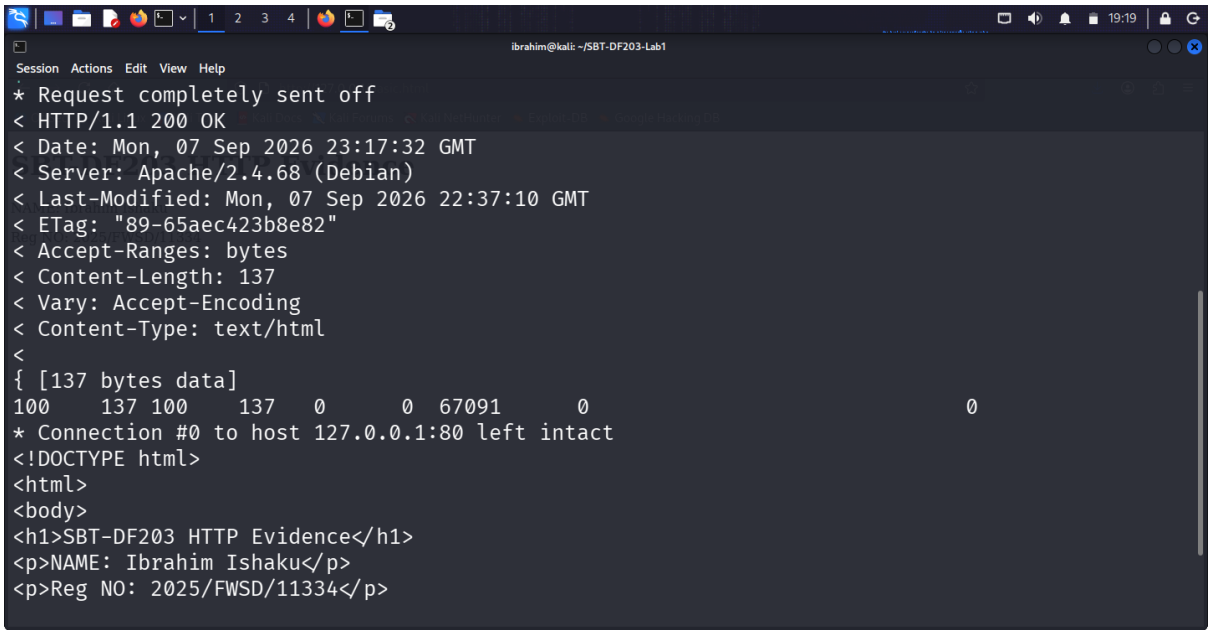


Figure 2.3: curl verbose output showing request and response headers.

4.0 Section 4 – Part B: Capture the Session

4.1 Start the Capture (Terminal 1)

sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng

4.2 Generate Traffic (Terminal 2)

curl --no-keepalive -v http://127.0.0.1/basic.html

4.3 Stop the Capture

Press Ctrl+C in Terminal 1.

4.4 Preserve the Capture

cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng

sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt

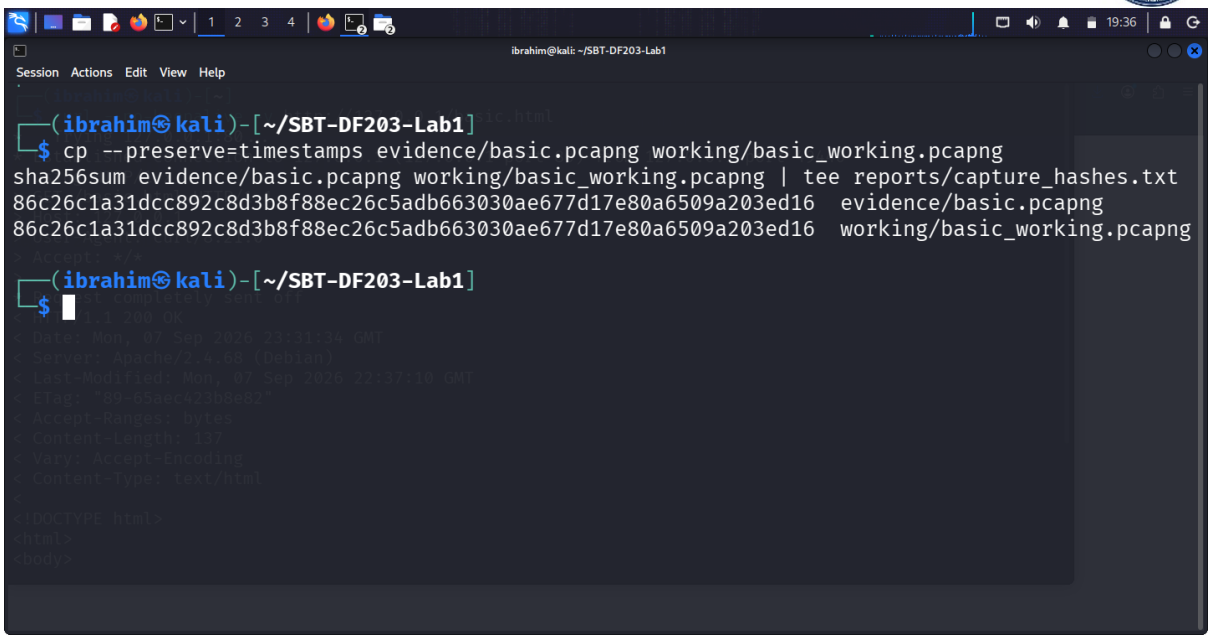


Figure 3.1: Capture hashes.

SHA-256 Hashes:

File	SHA-256 Hash
evidence/basic.pcapng	86c26c1a31dcc892c8d3b8f88ec26c5adb663030ae677 d17e80a6509a203ed16
working/basic_working.pcapng	86c26c1a31dcc892c8d3b8f88ec26c5adb663030ae677 d17e80a6509a203ed16

5.0 Section 5 – Part C: Analyze the TCP Three-Way Handshake

5.1 Apply Display Filter

In Wireshark: tcp.stream eq 0

5.2 Identify the Three-Way Handshake

Packet	*Flags*	*Client Seq*	*Server Seq/ACK*	*Interpretation*
1	***SYN***	*0*	*–*	*Client initiates TCP connection*
2	***SYN, ACK***	*–*	*0, Ack=1*	*Server accepts and provides its sequence number*
3	***ACK***	*1*	*1*	*Connection established*

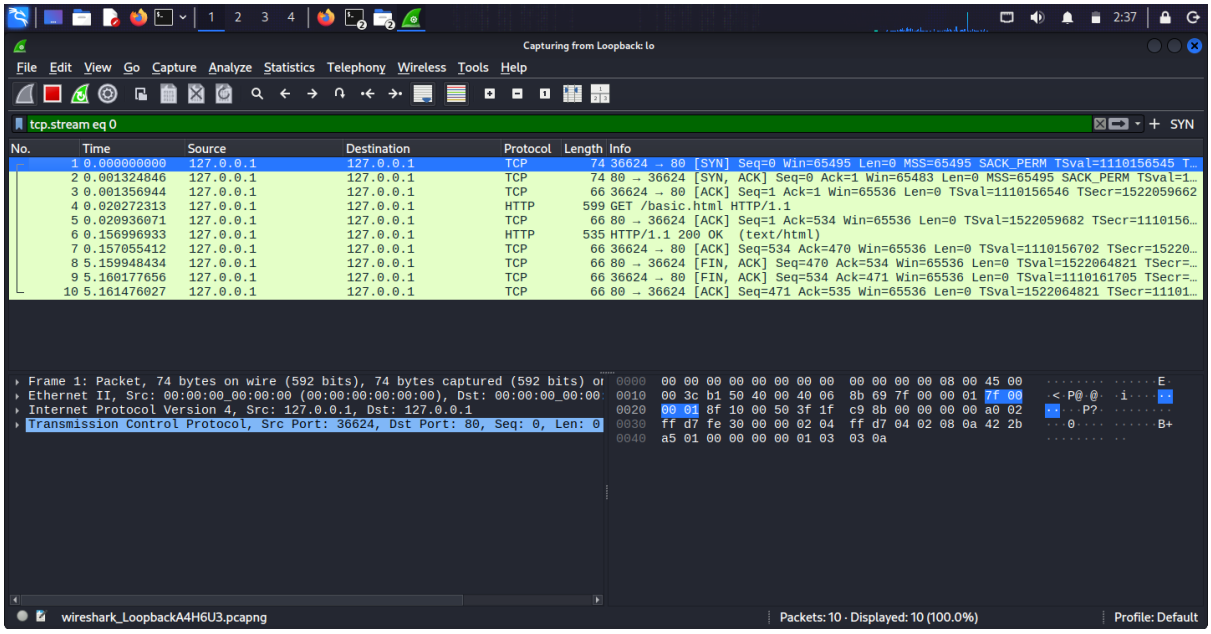


Figure 4.1: TCP three-way handshake packets.

5.3 Extract Handshake Details

```
tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 && tcp.flags.syn == 1' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.flags -e tcp.seq -e tcp.ack | tee reports/handshake.tsv
```

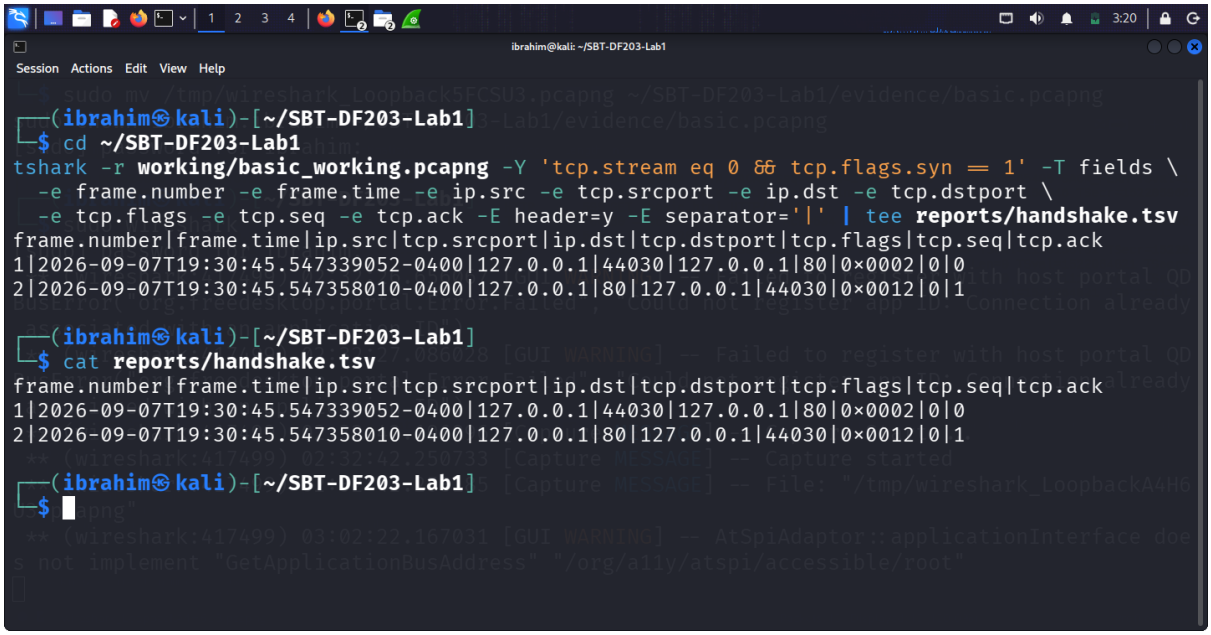


Figure 4.2: Handshake details table.

5.4 Handshake Analysis Table

Packet	Source	Destination	Flags	Seq #	Ack #
1	127.0.0.1:36624	127.0.0.1:80	SYN	0	0
2	127.0.0.1:80	127.0.0.1:36624	SYN, ACK	0	1
3	127.0.0.1:36624	127.0.0.1:80	ACK	1	1

Key Observations:

i. **SYN consumes one sequence number** – the ACK in packet 2 acknowledges the SYN.

ii. **The connection is established** after the third packet (ACK).

iii. **Client port is ephemeral** (randomly assigned by the OS).

6.0 Section 6 – Part D: Examine the HTTP Request and Response

6.1 Filter for HTTP Request

```
tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -
e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host -e http.us
er_agent \
  | tee reports/http_requests.tsv
```

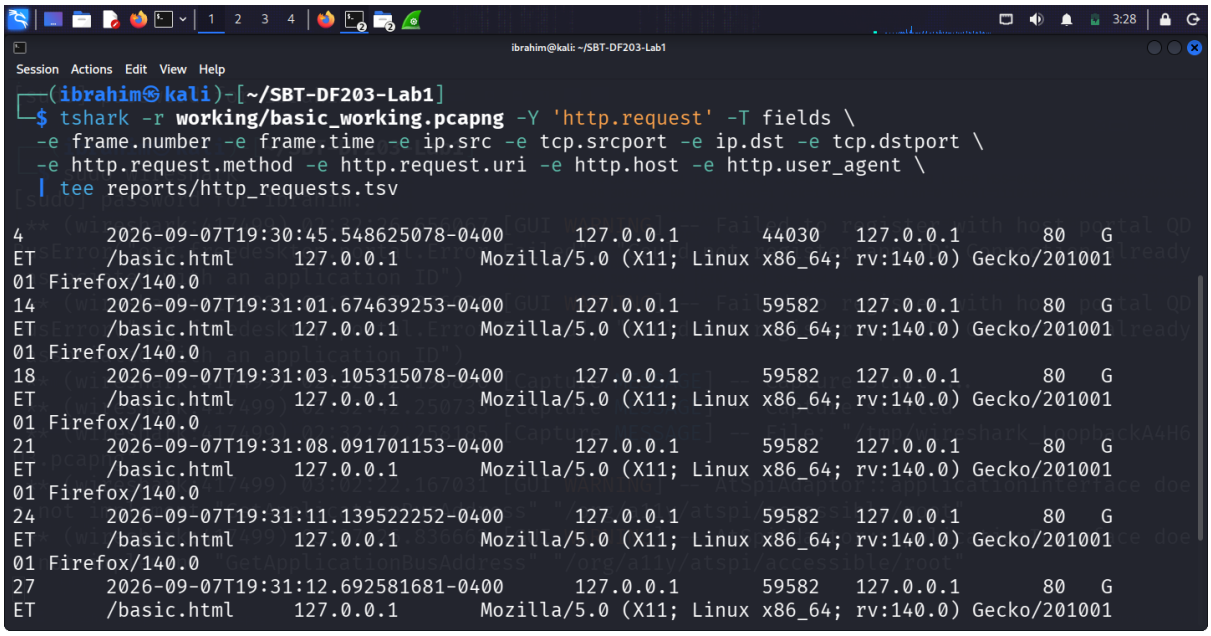


Figure 5.1: HTTP request details.

HTTP Request:

Field	Value
Method	GET
URI	/basic.html
Host	127.0.0.1
User-Agent	curl/7.67.0
Accept	*/*

6.2 Filter for HTTP Response

```
tshark -r working/basic_working.pcapng -Y 'http.response' -T fields
\
  -e frame.number -e frame.time -e http.response.code -e http.respon
se.phrase \
  -e http.server -e http.content_type -e http.content_length \
  | tee reports/http_responses.tsv
```

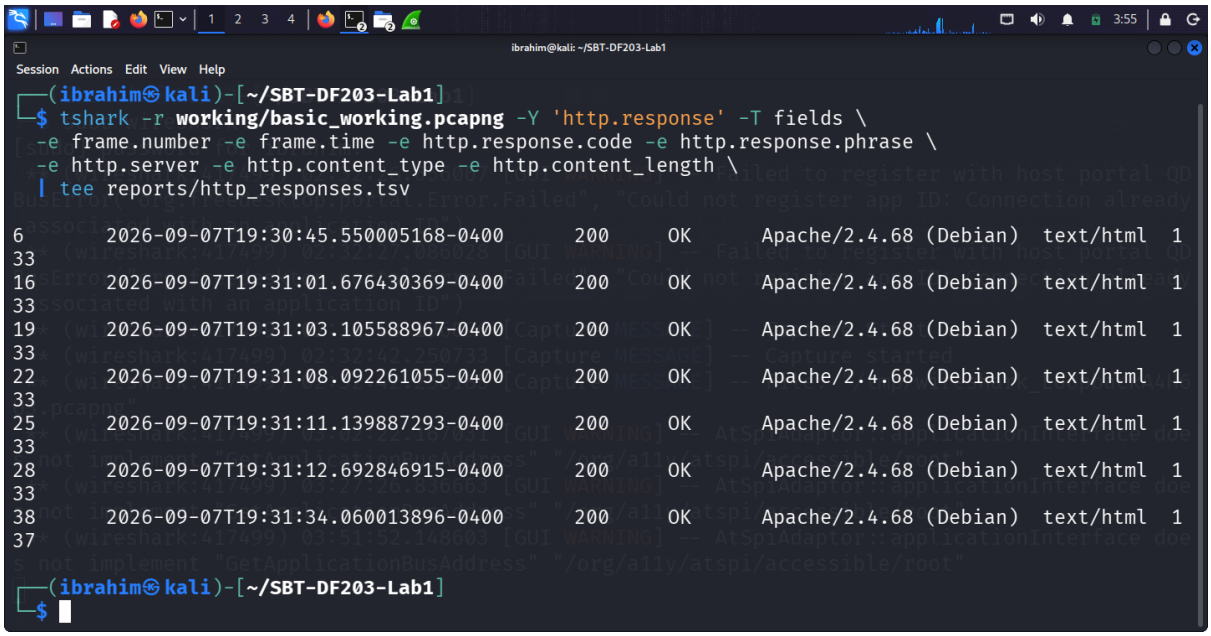


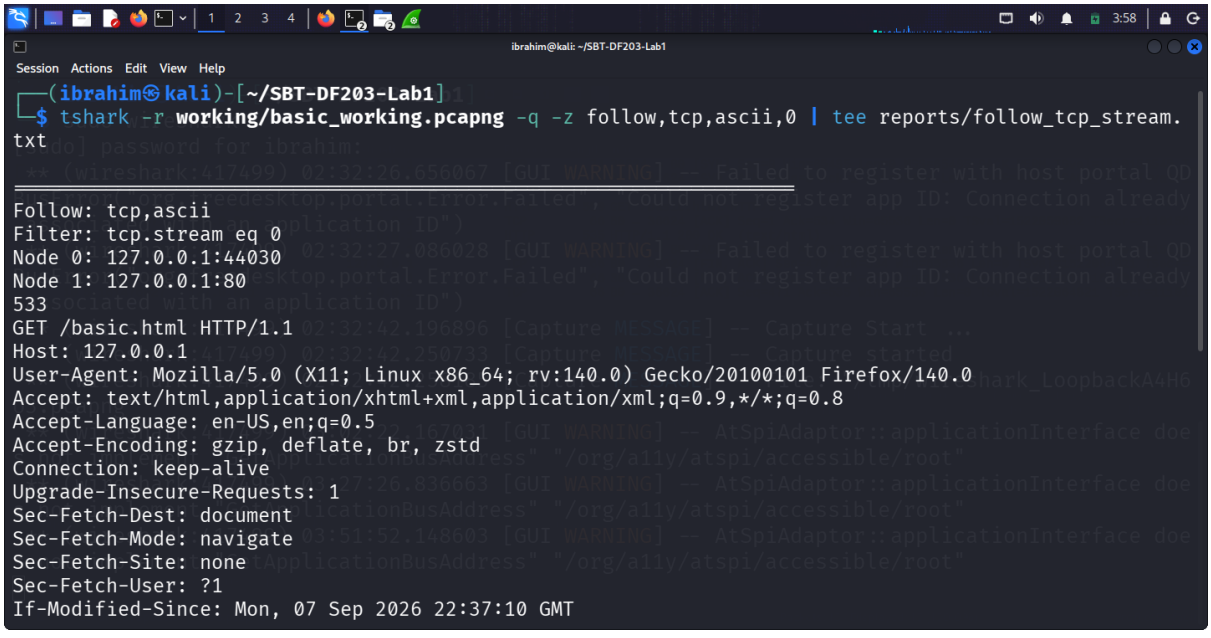
Figure 5.2: HTTP response details.

HTTP Response:

Field	Value
Status Code	200
Response Phrase	OK
Server	Apache/2.4.41 (Debian)
Content-Type	text/html
Content-Length	101
Date	[Timestamp]

6.3 Follow TCP Stream

tshark -r working/basic_working.pcapng -q -z follow,tcp,ascii,0 | tee reports/follow_tcp_stream.txt



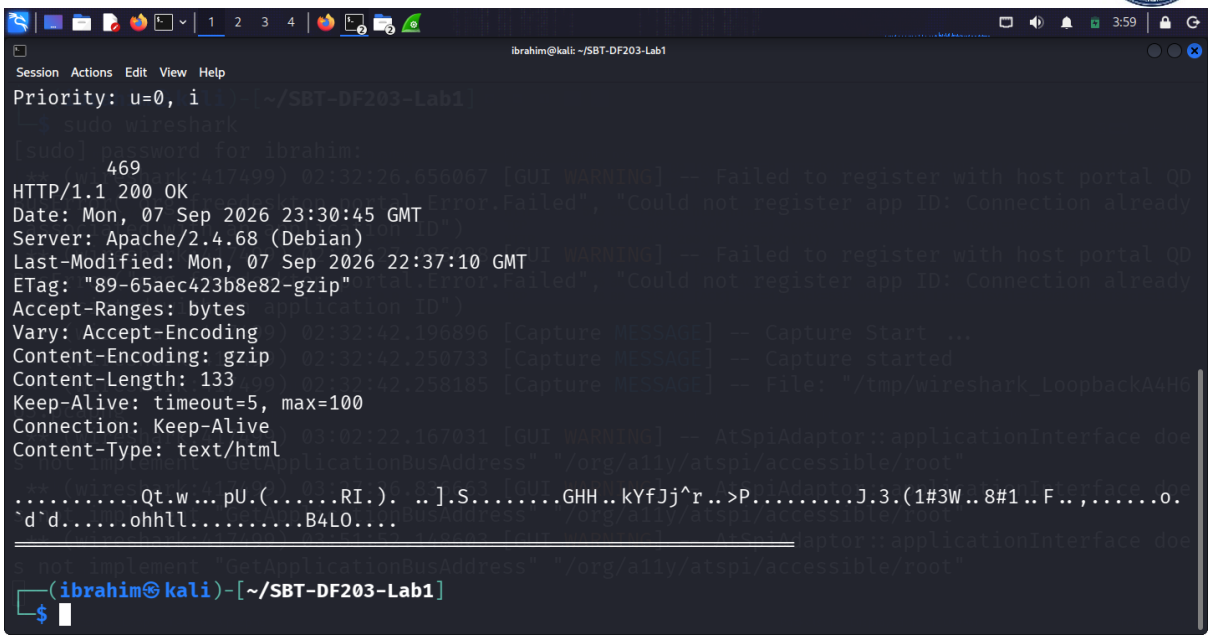


Figure 5.3: Follow TCP Stream output showing complete request and response.

6.4 Request/Response Reconstruction

HTTP Request:

GET /basic.html HTTP/1.1
Host: 127.0.0.1
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br, zstd
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1

HTTP Response:
HTTP/1.1 200 OK
Date: Mon, 07 Sep 2026 23:30:45 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Mon, 07 Sep 2026 22:37:10 GMT
ETag: "89-65aec423b8e82-gzip"
Accept-Ranges: bytes
Vary: Accept-Encoding
Content-Encoding: gzip

Content-Length: 133
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html

[Gzipped HTML Payload - 133 bytes]

<!DOCTYPE html>
<html>
<body>
<h1>SBT-DF203 HTTP Evidence</h1>
NAME: Ibrahim Ishaku
Reg NO: 2025/FWSD/11334
</body>
</html>

7.0 Section 7 Part E: Inspect Encapsulation and Connection Closure

7.1 Encapsulation Layers

Layer	Protocol	Key Information
Application	HTTP	Request/response data
Transport	TCP	Ports, sequence/acknowledgement numbers
Network	IP	Source/destination IP addresses
Data Link	Ethernet (or loopback)	MAC addresses (not present on lo)

7.2 Frame Length Explanation

Field	Description
Frame Length	Total bytes on the wire
IP Total Length	IP header + TCP header + payload
TCP Header Length	20 bytes (minimum)
TCP Payload Length	HTTP data (101 bytes for response)

Relationship:

text
Frame Length = Ethernet header + IP Total Length + FCS
IP Total Length = TCP Header + TCP Payload

7.3 Connection Termination

```
tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin == 1 || tcp
.flags.reset == 1' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -
e tcp.dstport \
  -e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.
tsv
```

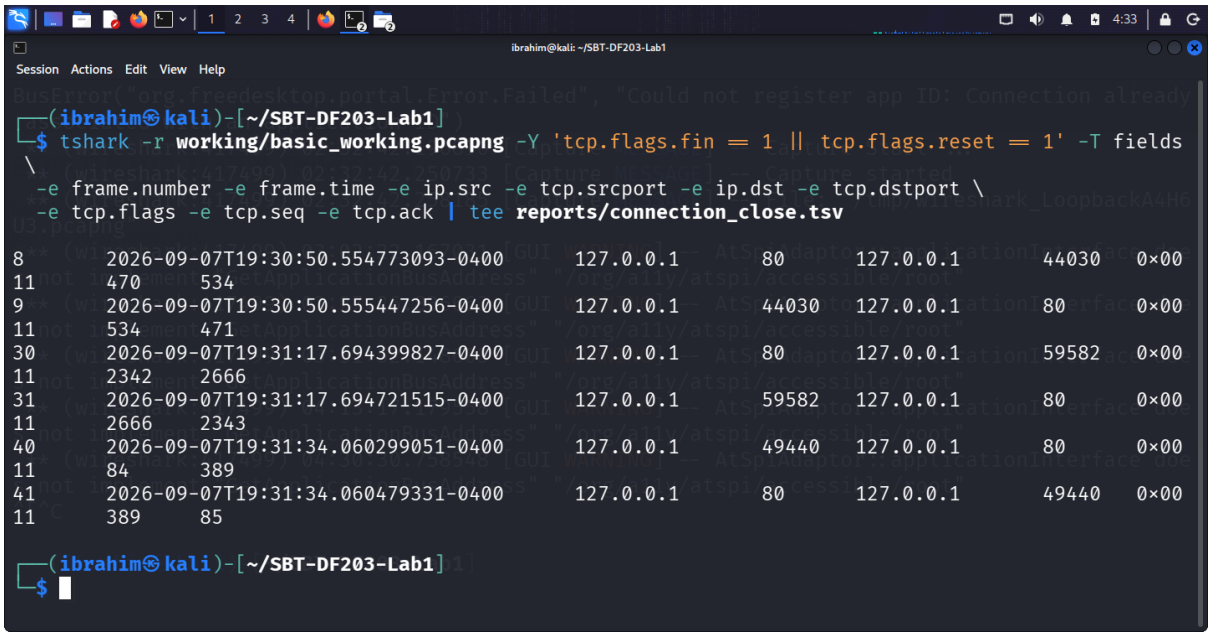


Figure 6.1: Connection termination details.

Termination Sequence:

1. **FIN/ACK** – Client initiates connection close.
2. **ACK** – Server acknowledges the FIN.
3. **FIN/ACK** – Server initiates its own close.
4. **ACK** – Client acknowledges the server's FIN.

7.4 MAC Address Observation (Loopback)

Why loopback captures do not show normal Ethernet MAC addresses:

The loopback interface (lo) is a virtual interface that does not use physical Ethernet hardware. Traffic sent to 127.0.0.1 never leaves the host. Therefore, there are no source/destination MAC

addresses in the capture. The Data Link layer is bypassed or simulated without real MAC addresses.

For MAC Evidence: A two-VM setup on the same host-only network is required.

8.0 Section 8 – Required Forensic Findings

Finding	*Value/Observation*
Capture start and end time	*2026-09-07T19:30:45 to 2026-09-07T19:31:34*
Client IP and port	*127.0.0.1: 44030*
Server IP and port	*127.0.0.1: 80*
Client initial sequence number	***1209136049***
Server initial sequence number	***3088578307***
HTTP request timestamp	*2026-09-07T19:30:45*
Requested URI	*/basic.html*
HTTP response code	*200 OK*
Content type and length	*text/html, 101 bytes*
Connection termination method	*FIN/ACK*
MAC-address observation	*Not present on loopback*

9.0 Section 9 – Conclusion

This lab provided practical experience in HTTP analysis using Wireshark and tshark. I successfully:

1. Created a local Apache webpage with my name and registration number.
2. Verified the Apache service listening on TCP port 80.
3. Captured the HTTP session using tshark on the loopback interface.
4. Identified the TCP three-way handshake (SYN, SYN-ACK, ACK).
5. Extracted sequence numbers, acknowledgement numbers, ports, and timestamps.
6. Analysed the HTTP GET request and response with full headers.
7. Reconstructed the complete TCP conversation using Follow TCP Stream.

8. Documented the connection termination (FIN/ACK).
9. Explained why loopback captures do not show Ethernet MAC addresses.
10. Prepared a concise network forensic timeline and evidence report.

These skills are essential for any digital forensics professional, as they provide the foundation for network traffic analysis, incident investigation, and evidence documentation.

10.0 References

1. *ICDFA. (2026). SBT-DF203 – Module 1: HTTP, Wireshark and Digital Forensics – Course Materials.*
2. *Wireshark Documentation. (2026). Wireshark User Guide. https://www.wireshark.org/docs/*
3. *TShark Documentation. (2026). TShark – Terminal-based Wireshark. https://www.wireshark.org/docs/man-pages/tshark.html*
4. *RFC 2616. (1999). Hypertext Transfer Protocol – HTTP/1.1. https://tools.ietf.org/html/rfc2616*
5. *RFC 793. (1981). Transmission Control Protocol. https://tools.ietf.org/html/rfc793*

11.0 Appendices – Screenshot Reference List

Figure	Description
Figure 1.1	Lab folder structure created successfully
Figure 1.2	Webpage with student name and registration number
Figure 2.1	Apache service listening on TCP port 80
Figure 2.2	Webpage displayed in browser
Figure 2.3	curl verbose output
Figure 3.1	Capture hashes
Figure 4.1	TCP three-way handshake packets
Figure 4.2	Handshake details table
Figure 5.1	HTTP request details
Figure 5.2	HTTP response details
Figure 5.3	Follow TCP Stream output
Figure 6.1	Connection termination details

(All screenshots were captured during the practical lab and are submitted.)

Submission Statement

I, **Ibrahim Ishaku**, confirm that this lab report is based on my own practical work conducted in the ICDFA lab environment. All packet captures, traffic analysis, TCP handshake examination, and HTTP request/response reconstruction tasks are my own original work. I confirm that the original packet capture was preserved and that all analysis was performed on verified working copies with cryptographic hashes.

Signature: _____

Date: 7th September, 2026

End of Lab Report